



LAYOFF DETECTION AI AGENT

A Complete Build Guide

From a manual LinkedIn hunt to a live, multi-agent app —

built with Claude Code, Gemini, SerpAPI · Apify · Perplexity, Supabase & Streamlit

Repository: github.com/adityasharmadotai-hash/amazing-ai-agents

Scroll down the repo and open **“Layoff AI Agent”** to clone it.

Contents

1. What You'll Build
2. Architecture at a Glance
3. The Toolbox — Every Tool & API
4. Step 0 — Prerequisites
5. Building It, Step by Step
6. Where to Get Every API Key
7. Parameters You'll Encounter
8. The Key Prompts (End-to-End)
9. Get the Code, Star It, Join the Community

1. What You'll Build

You'll build an AI agent that watches LinkedIn (and, optionally, the news) for layoff and "open to work" posts, reads every post with an LLM, keeps the people who match your target (role + location), and stores them as clean, contactable leads — with live cost tracking and a one-click dashboard. It runs as a single Streamlit web app you can host for free.

The origin story: I kept doing this by hand — searching for people who were just laid off, builders active in hackathons, and companies that were hiring. It worked, but it ate hours every week. So I turned the manual routine into a multi-agent system I now glance at **once a day**.

The funnel, in one line

Search providers → AI extraction → location/role filter → Supabase store → Streamlit dashboard

What "multi-agent" means here: several independent discovery agents (SerpAPI, Apify, Perplexity, Gemini search) run in parallel, then a separate extraction agent reads each result, and a location-enrichment agent fills in missing data. Each does one job well; the pipeline merges them.

2. Architecture at a Glance

Five layers, each swappable without touching the others:

1. **Discovery layer** — finds candidate posts. One or more providers run concurrently and results are de-duplicated by URL.
2. **Extraction layer** — Gemini reads each post + its URL slug and returns structured JSON (person, company, role, location, open-to-work, confidence).
3. **Enrichment layer** — when a post doesn't state a country, an optional profile scrape resolves it; Wiza can turn a profile into a verified work email.
4. **Storage layer** — Supabase (Postgres) stores every lead, de-duped on the post URL.
5. **Presentation layer** — a Streamlit dashboard (scan button, leads table, spend, history) plus a guided Settings page.

Design principle: the discovery providers are interchangeable. Run just SerpAPI (free) to start, then add Apify or Perplexity later by setting one environment variable — no code changes.

3. The Toolbox — Every Tool & API

Here is everything the project uses and exactly where to get it. You do NOT need all of them — the "Required" rows are the minimum; the rest are optional upgrades.

Tool	What it does	Cost	Get it at
Claude Code	AI pair-programmer that writes the app from plain-English prompts	Free / paid	claude.com/claude-code
Google Gemini (required)	Reads each post and extracts structured fields (the 'brain')	Free tier	aistudio.google.com/app/apik ey
Supabase (required)	Postgres database that stores the leads	Free tier	supabase.com
SerpAPI	LinkedIn discovery via Google's public index — easiest, free to start	Free 100/mo	serpapi.com

Apify	Full LinkedIn post scraping — higher volume, richer text	Paid / run	apify.com
Perplexity	Web-search discovery via the /search API	Paid	perplexity.ai (API)
Gemini Google Search	‘Deep-research’ style grounded search using your Gemini key	Free tier	same Gemini key
Wiza (optional)	Turns a LinkedIn profile into a verified work email	Paid	wiza.co
NewsAPI (optional)	Adds news articles as a second source	Free tier	newsapi.org
Streamlit	The web UI framework + free public hosting	Free	streamlit.io

Lesson learned: the providers disagree — on purpose

I ran the exact same search through SerpAPI, Apify, Perplexity, and Gemini’s Google-Search grounding. Every one returned a different set of people. None is a superset of the others:

- **SerpAPI** — whatever Google has publicly indexed. Great breadth, slightly delayed, misses login-walled posts.
- **Apify** — scrapes the live post feed. Highest volume and richest text, but paid per run.
- **Perplexity** — its own web index + ranking. Surfaces posts the others miss; different noise profile.
- **Gemini search** — grounded generation; good at phrasing-based recall, weakest at exact post URLs.

Takeaway: don’t bet on one source. The app supports an “all” mode that runs every provider you have a key for, concurrently, and merges + de-dupes the results. Start with SerpAPI, then layer the others in.

4. Step 0 — Prerequisites

6. A computer with Python 3.10+ and Git installed.
7. Claude Code installed (see the command below).

```
npm install -g @anthropic-ai/claude-code
# then, in an empty project folder:
claude
```

8. A GitHub account (to host the code and deploy from it).
9. Free accounts for the two required services: Google AI Studio (Gemini) and Supabase. Add SerpAPI to actually pull posts.

Why Claude Code: you describe each layer in plain English and Claude Code writes, runs, and debugs the Python for you. The prompts in Section 8 are the exact instructions you give it — you don’t need to type the code by hand.

5. Building It, Step by Step

Each step is one focused conversation with Claude Code. Build in this order — every layer works on its own before you add the next.

Step 1 · Scaffold the project

Create the folder skeleton and a config module that reads all settings from environment variables (so nothing is hard-coded and secrets never touch the code).

```

agent/          # the pipeline (no UI)
  config.py     # reads env vars, defaults, target filters
  llm.py        # single Gemini helper -> complete_json()
  extract.py    # post text -> structured JSON
  pipeline.py   # collect -> extract -> store (one scan)
  store.py      # Supabase read/write
  usage.py      # per-scan cost tracking
  sources/      # discovery providers (serpapi, apify, ...)
views/          # streamlit pages (dashboard, settings)
streamlit_app.py # router + theme

```

Step 2 • Discovery layer (find the posts)

Start with SerpAPI. It searches Google for public LinkedIn posts matching a set of phrases people actually write when they're laid off. Keep providers behind one function so you can add Apify/Perplexity later without touching the rest of the pipeline.

The default query set mixes employee language with layoff keywords — this is what makes small startups (not just the big names) surface:

```

"my last day at"          "I was impacted by the layoffs"
"today was my last day"   "affected by the recent layoffs"
"my role was eliminated"  "part of the layoffs at"
"laid off"                "reduction in force"
"open to work" ("laid off" OR layoff OR "let go")
(#layoffs OR "layoffs at" OR "laying off" OR "cutting jobs")

```

Step 3 • Extraction layer (the brain)

Every raw post goes to Gemini with a strict instruction to return JSON only. One helper module talks to the LLM so there's exactly one place to change models or keys. The full prompt is in Section 8 — it's the single most important part of the whole project.

Pro tip: feed the model the post's URL slug, not just the snippet. A URL like `.../posts/jane_i-was-impacted-by-layoffs-at-xbox` recovers the company and signal even when the visible snippet is thin.

Step 4 • Filter & locate

The extractor returns a country/location. A filter keeps only people in your target locations — with a deliberate 'benefit of the doubt' rule: posts with an unknown location still pass (LinkedIn search is already biased to your region), which recovers a LOT of leads. Confirmed other-country people are dropped.

Step 5 • Storage (Supabase)

Create a free Supabase project and run the provided SQL to make a leads table keyed on the post URL (so re-scans de-dupe automatically). The app talks to Supabase over its REST API — no heavy client library needed.

Step 6 • Cost tracking

Every paid call (Apify scrapes, Gemini tokens, SerpAPI searches, Perplexity requests) is counted during a scan and priced from rates in config. This powers the dashboard's Expenses panel so you always know your spend per scan and cumulatively.

Step 7 · The Streamlit dashboard

Streamlit turns Python into a web app with no front-end code. `streamlit_app.py` is a router that sets the theme and mounts two pages via `st.navigation`. The dashboard has a Scan button, a leads table, an Expenses panel, and a scan history tab.

Run it locally with:

```
pip install -r requirements.txt
streamlit run streamlit_app.py
```

Step 8 · The Settings page

A guided page where every API key has step-by-step ‘how to get it’ instructions, and a picker chooses the discovery path (SerpAPI vs Apify). Users configure the whole agent without editing code.

Step 9 · Deploy free on Streamlit Community Cloud

10. Push the repo to GitHub.
11. Go to share.streamlit.io → New app → pick your repo and `streamlit_app.py`.
12. Open Settings → Secrets and paste your keys in TOML form (never commit them). Example:

```
GEMINI_API_KEY = "your-key-here"
SUPABASE_URL = "https://xxxx.supabase.co"
SUPABASE_SERVICE_KEY = "your-service-key"
SERPAPI_KEY = "your-serpapi-key"
LINKEDIN_SOURCE = "serpapi"
```

13. Deploy. The app is live at a public URL and redeploys automatically on every git push.

6. Where to Get Every API Key

Key	How to get it
GEMINI_API_KEY	aistudio.google.com/app/apikey → sign in → Create API key → copy the Alza... value.
SUPABASE_URL	supabase.com → new project → Project Settings → Data API → copy Project URL.
SUPABASE_SERVICE_KEY	Supabase → Project Settings → API keys → reveal & copy the service_role key (keep secret).
SERPAPI_KEY	serpapi.com → sign up free → Dashboard → copy your private API key.
APIFY_TOKEN	apify.com → Settings → Integrations → API tokens → copy your personal token.
PERPLEXITY_API_KEY	perplexity.ai → API settings → generate a key (uses the /search endpoint).
WIZA_API_KEY (optional)	wiza.co → Settings → API → generate a key (enables email enrichment).
NEWSAPI_KEY (optional)	newsapi.org → register free → copy the key shown on your account page.

Security: keys live only in your local `.env` (git-ignored) or in Streamlit’s Secrets box — never in the code. This guide contains no real keys, and you should never paste yours into a document, screenshot, or chat.

7. Parameters You’ll Encounter (and What They Do)

Everything is tunable from environment variables (or the Settings page). These are the knobs that matter most:

Setting	Default	What it controls
---------	---------	------------------

LINKEDIN_SOURCE	all	Which discovery provider(s): all serpapi apify perplexity gemini. 'all' merges every provider you have a key for.
LINKEDIN_RECENCY_DAYS	3	Only keep posts from the last N days (freshness).
LINKEDIN_RESULTS_PER_Q	30	Results pulled per query per provider. Higher = more coverage & more cost.
APIFY_MAX_KEYWORD_VARIANTS	8	Caps how many paid Apify searches run per scan.
TARGET_LOCATIONS	SF, California	Comma-tolerant list to filter by. Blank = worldwide.
LOCATION_INCLUDE_UNKNOWN	true	Keep leads whose location couldn't be determined (recovers many leads).
LOCATION_IN_SEARCH	false	Put location into the query (precise, fewer hits) vs. use it only as a filter.
TARGET_TITLES	36 SWE roles	Role labels to tag/keep (e.g. Data Scientist, ML Engineer).
GEMINI_MODEL	gemini-2.5-flash	Cheap & fast; switch to a pro model for higher quality.
ENRICH_LOCATION	true	Scrape the profile to resolve an unknown country (one extra scrape).
SCAN_INTERVAL_HOURS	4	Suggested cadence if you automate scans.

Gotchas I hit (so you don't have to)

- **Commas in place names.** "San Francisco, California" must NOT be split on the comma — parse locations on newline/pipe only.
- **Import a heavy SDK lazily.** The Gemini library pulls in C extensions that crashed Streamlit Cloud's startup health-check. Import it inside the first function call, not at module top.
- **Retry only on bad JSON.** If the model returns non-JSON, retry; if the API rejects the key/model, fail loudly — don't silently return 0 leads.
- **Zero leads ≠ no layoffs.** Surface the real reason (usually a bad key or an over-strict location filter) in the UI.

8. The Key Prompts (End-to-End)

These are the prompts that actually build and run the agent. Nothing here contains a secret — all keys are read from the environment at runtime.

8A · Claude Code build prompts

Give these to Claude Code, one per step. They're intentionally plain-English — Claude Code writes and runs the code.

Prompt 1 — scaffold

```
Build a Python project called "layoff detection agent". Structure it as an
"agent" package (config, llm, extract, pipeline, store, usage, and a sources/
sub-package) plus a Streamlit UI. Read ALL configuration from environment
variables via a config.py that has a refresh() function. No secrets in code.
```

Prompt 2 — discovery

```
Add a discovery layer that finds public LinkedIn posts about layoffs and "open to work". Start with a SerpAPI provider that Google-searches site:linkedin.com/posts for a list of layoff phrases, paginates, and returns {url, text, source, profile_url}. Put every provider behind one search_linkedin_posts() function so I can add Apify and Perplexity later and merge them, de-duped by URL.
```

Prompt 3 — extraction

```
Add an extract.py that sends each post to Gemini and returns STRICT JSON with: is_layoff, is_individual, poster_role, company, person_name, role_hint, country, is_us, location, headcount, event_date, open_to_work, summary, confidence. Never invent facts – use null when unknown. Also feed the model the keywords recovered from the post URL slug. One shared llm.py talks to Gemini and parses JSON, retrying only when the reply isn't valid JSON.
```

Prompt 4 — filter, store, cost, UI

```
Keep only people in TARGET_LOCATIONS, but let unknown-location posts pass. Store every lead in Supabase, de-duped on the post URL. Track the cost of each scan (Apify, Gemini tokens, SerpAPI). Then build a Streamlit dashboard with a Scan button, a leads table, an Expenses panel, and a scan-history tab, plus a Settings page where each API key has step-by-step "how to get it" instructions.
```

8B • The extraction prompt (the core of the agent)

This is the exact instruction that turns a messy post into structured data. System message first, then the per-post template:

System

```
You are a precise information-extraction engine for a recruitment team that tracks layoffs. You are given the text of a social/news post. Decide whether it describes a real layoff event or a person affected by one, then extract structured fields. Never invent facts – use null when unknown. Respond ONLY with a JSON object matching the requested schema.
```

Per-post template

```
Analyze the post below.
```

```
Return JSON with exactly these keys:
```

- is_layoff (boolean): true if this describes an actual layoff event OR a person affected by one (laid off, role eliminated, RIF, "open to work" due to a layoff, or a company announcing/reporting layoffs).
- is_individual (boolean): true if the post is by/about a specific laid-off PERSON (a potential candidate); false if a company/news/opinion post.
- poster_role (string): one of "employee", "recruiter", "founder", "company", "news", or "other".
- company (string|null): the employer where the layoff happened. Extract it even from casual phrasing ("my last day at ACME", "impacted by layoffs at Retell AI"). Return the cleanest common company name.
- person_name (string|null): the affected individual, if the poster IS them.
- role_hint (string|null): the raw job title / function, verbatim.
- country (string|null): normalized country. Infer from any location text, "based in", city/state names, or context. null only if truly no signal.


```
- is_us (boolean): true only if the person/company is US-based.
- location (string|null): the most specific place stated or implied.
- headcount (integer|null): number of people laid off, if stated.
- event_date (string|null): ISO date (YYYY-MM-DD), inferred from "yesterday",
  "last week", etc.
- open_to_work (boolean): true if they signal they are job-seeking.
- summary (string): one neutral sentence (max 30 words).
- confidence (number): 0.0-1.0 – how confident this is a real layoff post.
```

Pay special attention to LOCATION and EVENT_DATE. Use BOTH the snippet and the URL-derived keywords below.

```
POST SOURCE URL: {url}
URL KEYWORDS: {slug}
POST TEXT:
""
{text}
""
```

8C · The discovery prompt (Perplexity / Gemini search)

When a provider does the searching itself (Perplexity or Gemini grounding), this is the instruction it gets:

```
Use web search to find recent PUBLIC LinkedIn posts where an individual says
they were laid off / impacted by layoffs / let go and are open to work.
```

```
Search intent: {your phrase}
Only include posts from roughly the last {N} days.
```

```
Return STRICT JSON only: an array of objects, each with:
  "url": the full post URL starting with https://www.linkedin.com/posts/
  "text": one sentence with the person's name, role, company, layoff signal.
```

Rules:

- Only real URLs you actually found. Never guess or fabricate a URL.
- Skip company announcements and news – only individuals' posts.
- Output the JSON array and nothing else.

9. Get the Code, Star It, Join the Community

The entire project is open source. Clone it, run it, and make it yours:

14. Open the repo: github.com/adityasharmadotai-hash/amazing-ai-agents
15. Scroll down the README and find “**Layoff AI Agent**” (a.k.a. Layoff Detection Agent).
16. Clone it and follow that folder’s README to run locally, then deploy free on Streamlit Cloud.

If it helps you: ★ star the repo, 🔄 share it with a friend who’s hiring or job-hunting, and open an issue with ideas.

Join the community: If you haven’t joined my AI WhatsApp community yet, come build with us — it’s where I share new agents, prompts, and walkthroughs. Invite link: [PASTE YOUR AI WHATSAPP COMMUNITY INVITE LINK HERE]

More from me: [Follow on LinkedIn](#) · [YouTube](#)